Reg N° : 23300106I
Mobile application systems and design
Computer and software Engineering
Year 3
25 March 2026

Question 1 : Dependencies that I used :

* Sqflite : Provide a plugin for flutter application to access and manage a local SQlite database. it provide SQlite support with full CRUD operations.
it acts as a bridge between dart code of flutter app and native SQlite database engines on android and IOs platform.

* Path_Provider dependency in flutter : is a plugin designed to Provide cross-platform access to commonly used file system locations on the host operating system. it locates the correct local storage directory per platform.

Why SQLite is necessary ?

SQlite is needed because it stores data on device with no internet required, supports structured queries and persists data even after the app closes.

Question 2 : Handling database exceptions :

Handling database exceptions effectively requires a layered approa using try-catch blocks, transaction management and comprehensive logging to ensure data integrity and application stability.

① Handling database not initialized :

This happens when your app tries to use a database before it is created or connected

Scanned with CamScanner

How to handle :

- check if database connection exist before performing operations
- initialize the database at app startup
- use try- catch block to catch connection errors.

② Handling Insert / update / delete Errors

These errors occurs due to constraints , syntax issues or connection failure

How to handle :

- wrap database operation in try- catch
- use transaction for safety
- Roll back if something fails

③ Handling invalid / corrupted data

This include wrong format, null values where not allowed or unexpected data types

How to handle :

- Validate input before storing in database
- use constraints in database (NOT NULL, UNIQUE)
- Sanitize inputs to avoid corruptions

Scanned with CamScanner

Question 3 : SQLite in flutter ?

SQLite is a lightweight, local database used to store data on devices. it's commonly used because it works offline, stores structured data in tables and ideal for mobile apps.

Database vs Table

Database refers to a structured set of data that holds data. they are .db file and contains multiple tables.

Table collection of related data organized in rows and columns. ex : Spreadsheet

CRUD operations :

CRUD stands for Create, Read, update, Delete that represent essential operation of SQL and NoSQL database.

- Insert
- Select
- update
- Delete

How flutter interacts with database asychronously

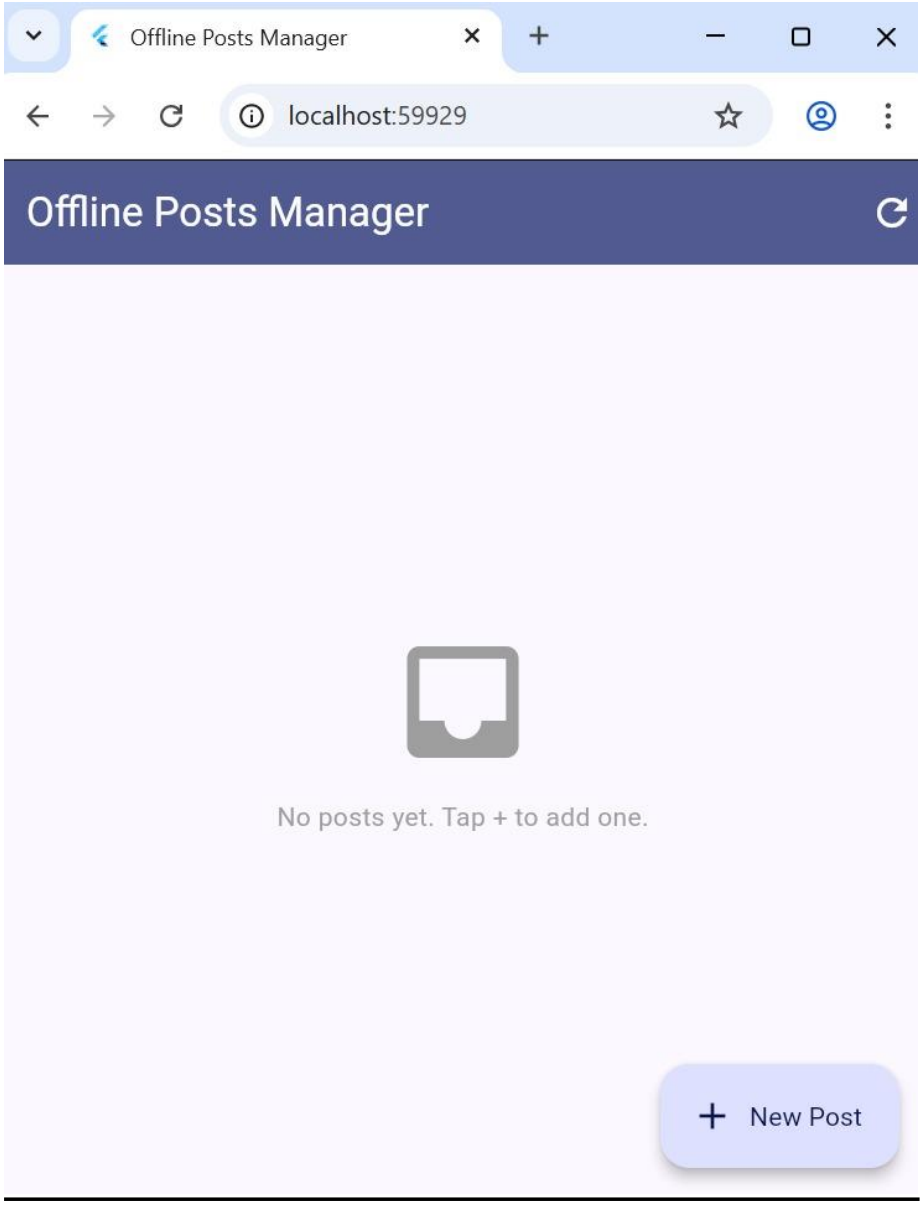
Flutter interacts with a database asychronously using sqflite and dart's asyn features.

This ensures that database do not block the user interface

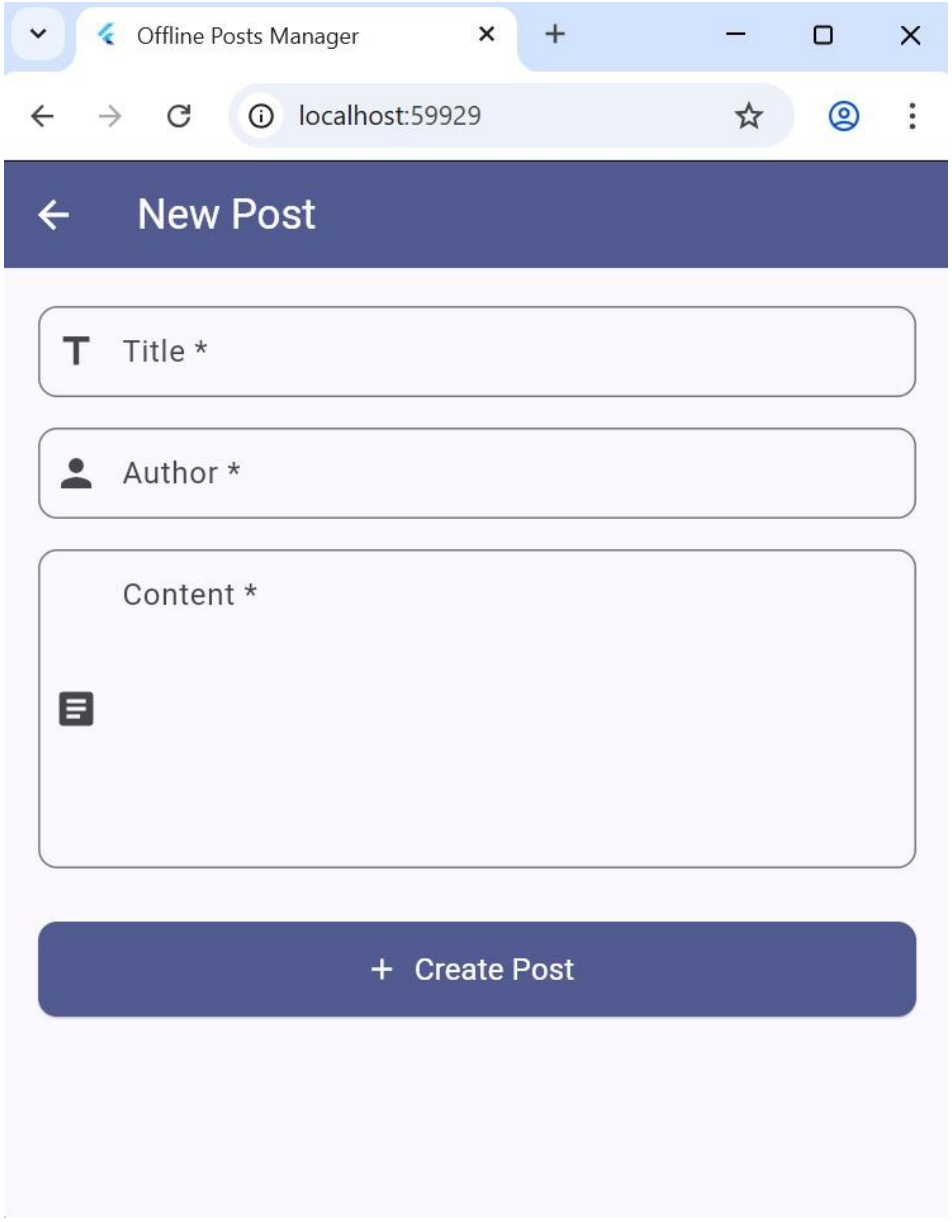
Flutter interacts with sQlite asychronously using async/ await so the UI never freezes while waiting for DB operations.

Scanned with CamScanner

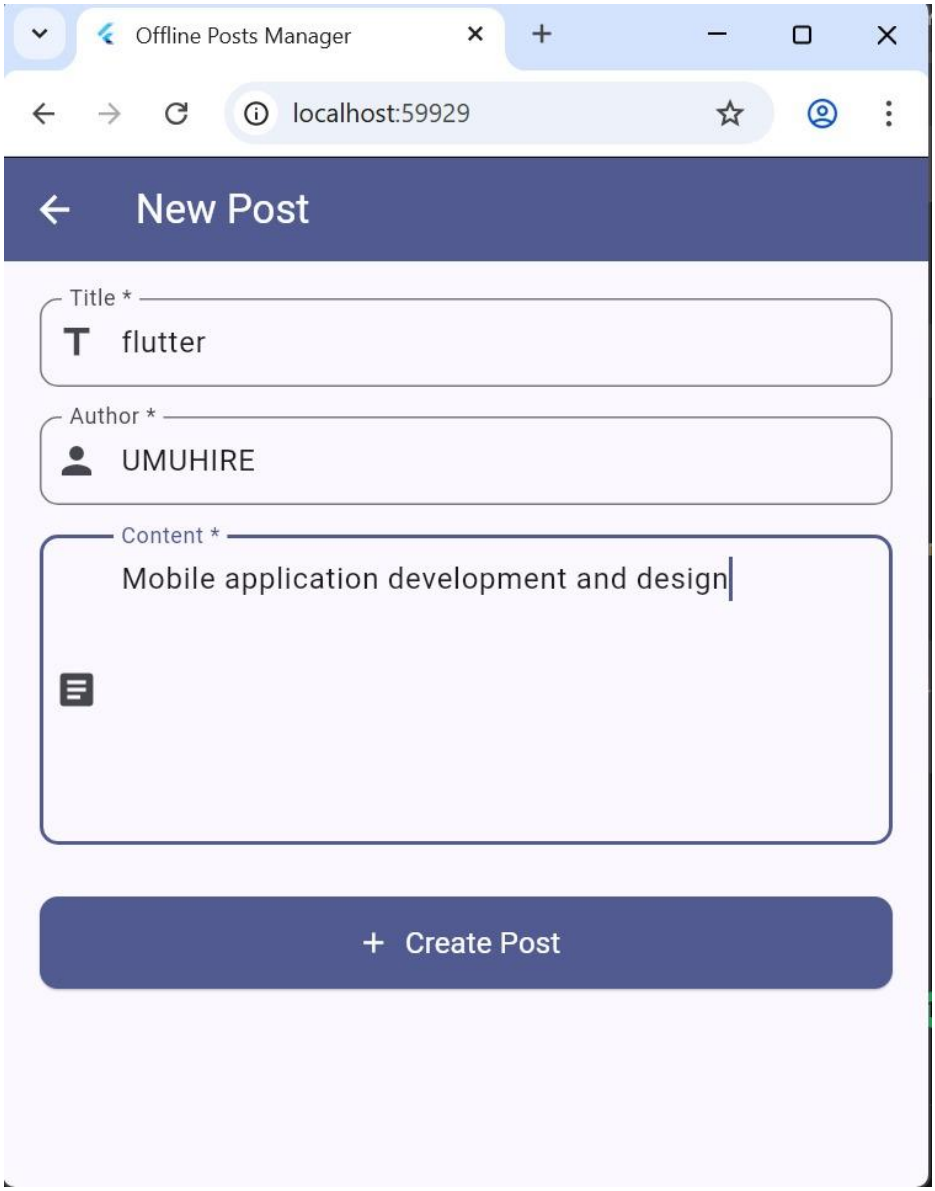
OUTPUT SCREENSHOOTS



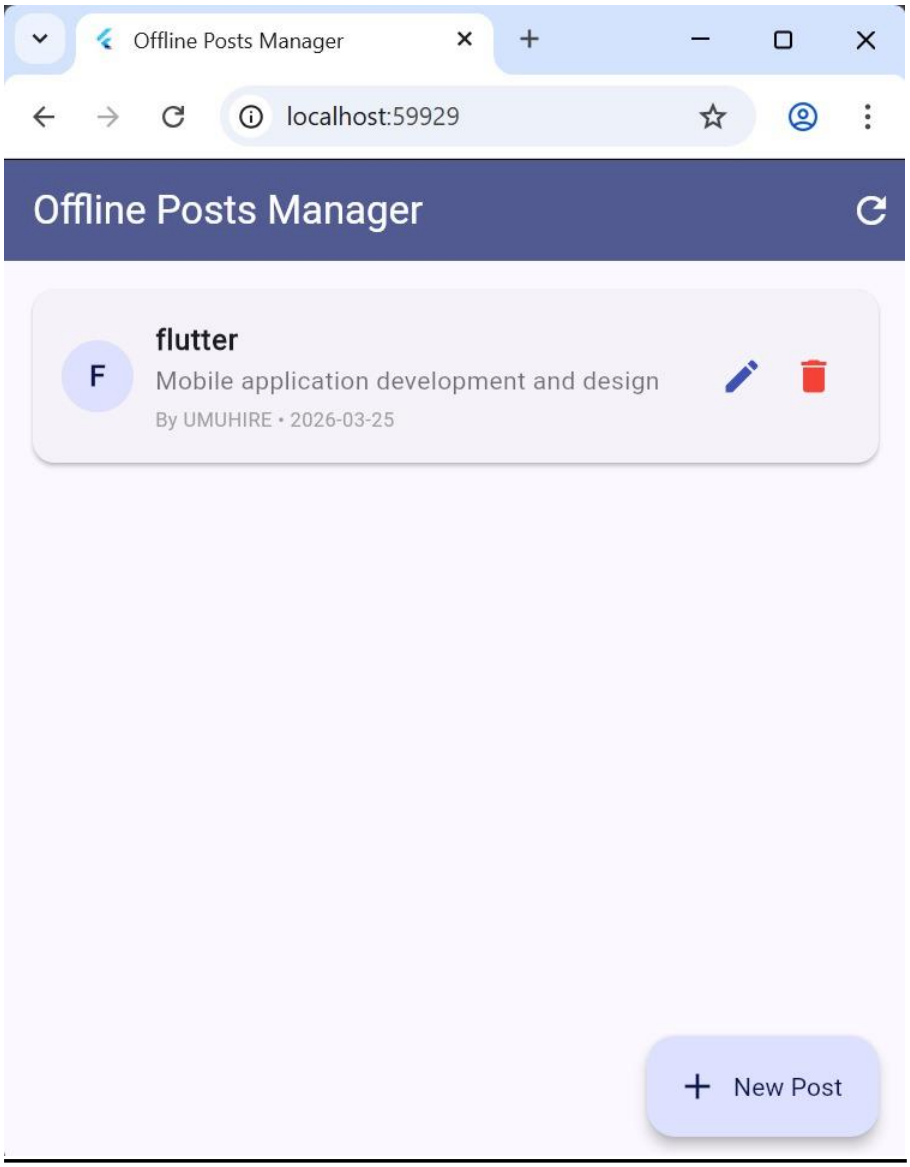
This is the initial home screen of the Offline Posts Manager app when launched for the first time or when no posts have been added yet. A floating action button labeled "+ New Post" is visible at the bottom right, which the user clicks to navigate to the post creation form.



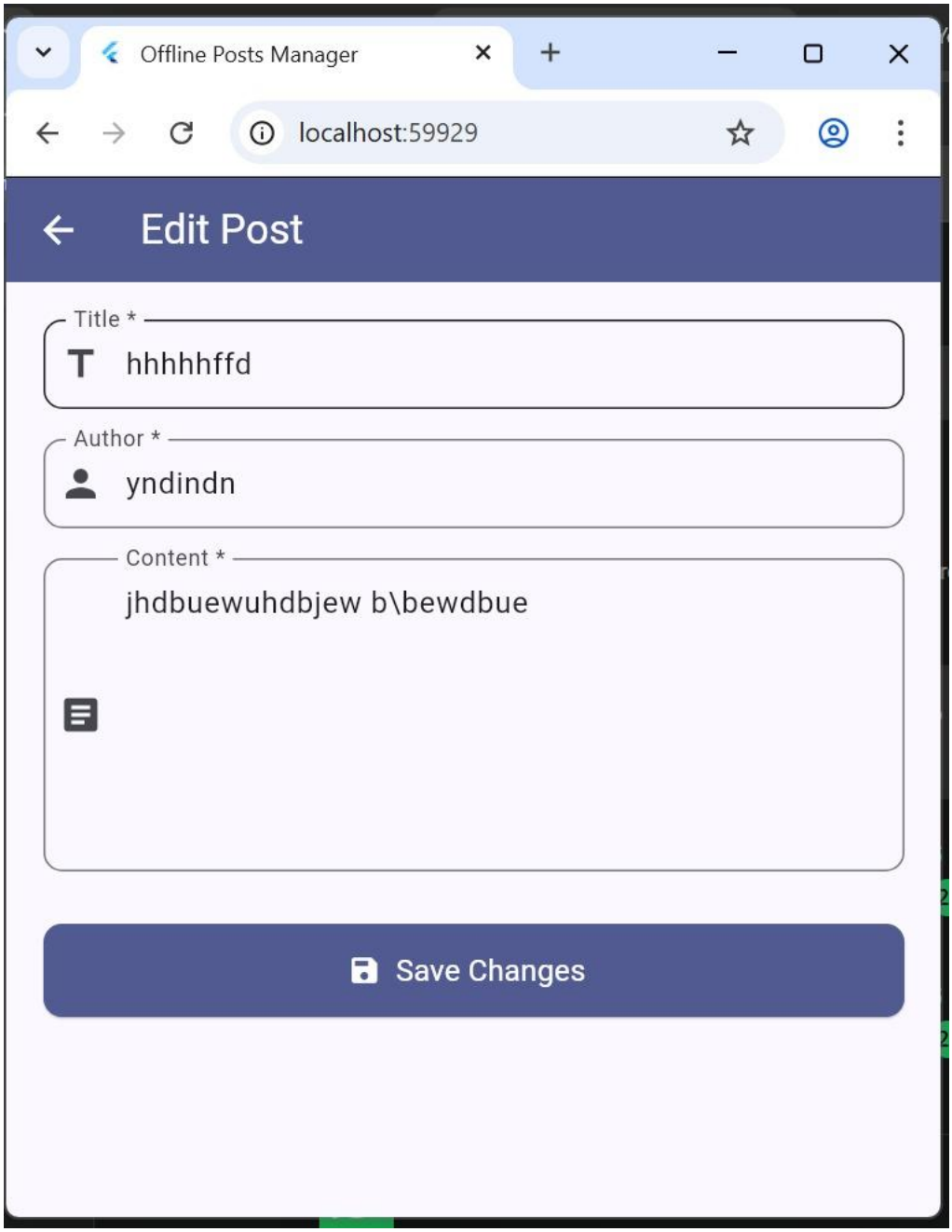
This screen shows the "New Post" form before any data has been entered. It is accessed by tapping the "+ New Post" action button on the home screen. The form contains three input fields: Title, Author, and Content. A "Create Post" button is displayed at the bottom of the form.



This screenshot demonstrates the New Post form after the user has filled in all required fields. The Title field contains "flutter", the Author field contains "UMUHIRE", and the Content field contains "Mobile application development and design". Once the user taps the "Create Post" button, the app calls the SQLite INSERT operation via the DatabaseHelper class to save the post locally. The data is persisted to the local SQLite database and remains available even without an internet connection.

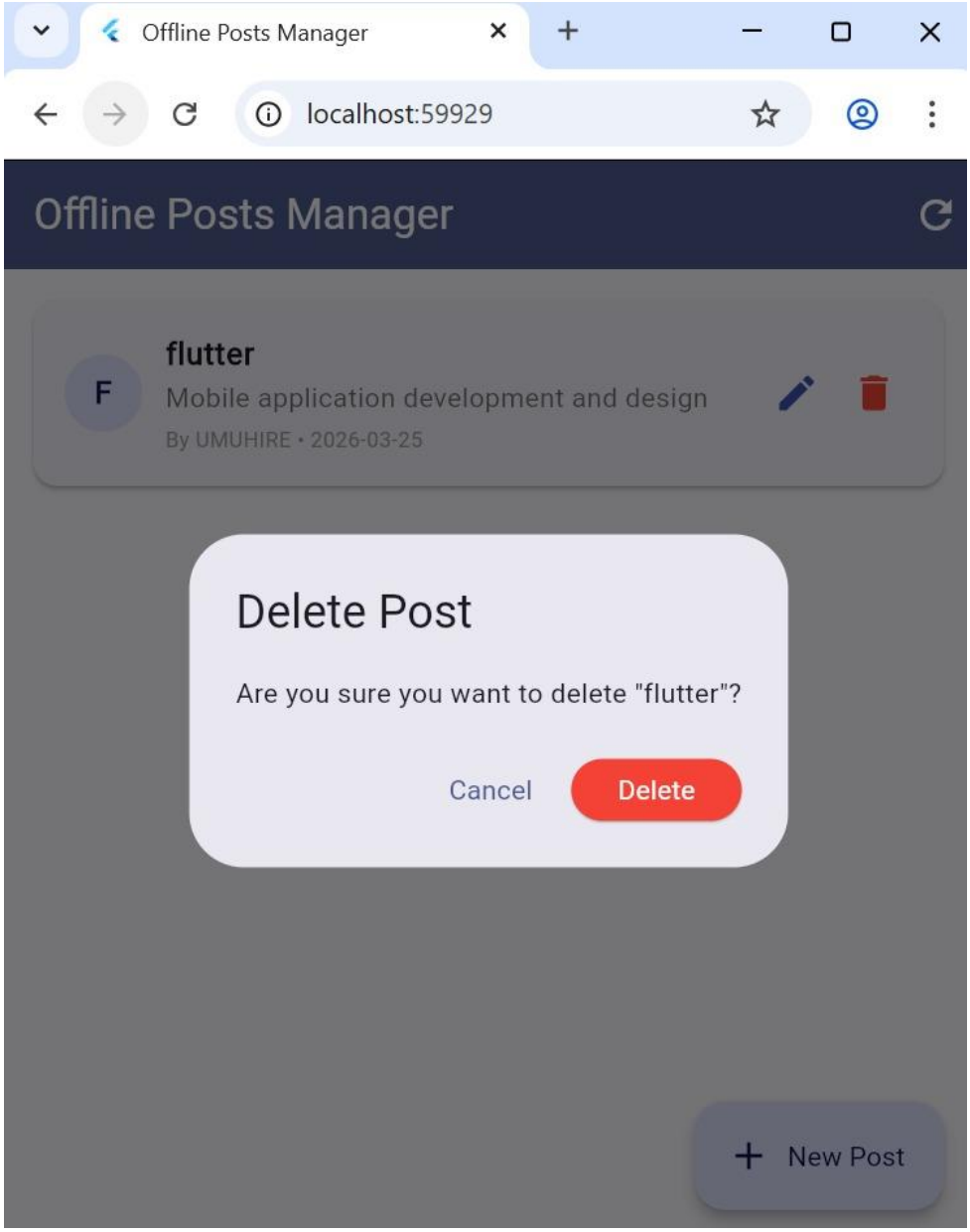


This screen shows the main posts list after a post has been successfully created and saved to the SQLite database. The post titled "flutter" is displayed. Below the title, a preview of the content "Mobile application development and design" is shown, followed by the author and date metadata: "By UMUHIRE • 2026-03-25". Two action icons appear on the right side of each card one for editing and another one for deleting. The user can also tap anywhere on the card to view full post details.



This screen shows the "Edit Post" form, which is loaded when the user taps the pencil (edit) icon on a post. The user can modify any of these

fields and tap "Save Changes" to update the record. The app performs an SQLite UPDATE operation using the post's unique ID to overwrite the existing record in the database.



This screenshot shows the delete confirmation dialog that appears when the user taps delete icon. The dialog asks "Are you sure you want to delete flutter?" to prevent accidental deletion. It presents two options: a "Cancel" (which dismisses the dialog and keeps the post) and "Delete" button (which confirms the deletion). Upon confirmation, the app calls the SQLite DELETE operation to permanently remove the post from the local database.